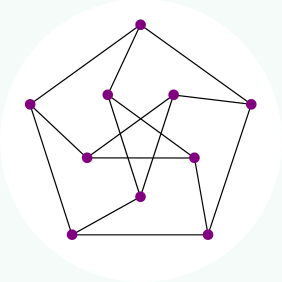


# AdaByron 2026 - Regional de Andalucía

Soluciones de los problemas

# Clasificación de los problemas

| Problema | Categoría |
|---|---|
| A - Tablero con Fichas | Ad-hoc |
| B - Encriptación matricial | Ad-hoc |
| C - El Circo de la Luna | 2-SAT + Búsqueda binaria |
| D - Tunelando el futuro | Grafos |
| E - El Número Misterio | Ad-hoc |
| F - El historial de AdaByron | Fenwick Tree |
| G - La muralla del reino | búsqueda binaria sobre solución |
| H - Control en Laboratorios | Ad-hoc |
| I - Numble | Interactivo |
| J - AByronOS | Pilas y Colas |
| K - Balanceando la Banda | Flujo Máximo |
| L - Descubriendo Genómicos | Ventanas deslizantes |
| M - La cascada de copas | PD |

# A. Dibujando tableros

| Envíos | Válidos | % éxito |
|---|---|---|
| 40 | 32 | 80 % |

# A. Dibujando tableros — Resumen

## ¿Qué se pide?

Dado un listado de piezas con coordenadas, imprimir el estado final de un tablero de ajedrez de $8 \times 8$.

# A. Dibujando tableros — Resumen

## ¿Qué se pide?

Dado un listado de piezas con coordenadas, imprimir el estado final de un tablero de ajedrez de $8 \times 8$.

- Casillas vacías en Unicode:
  - clara: `\u25FB` (WHITE SQUARE)
  - oscura: `\u25FC` (BLACK SQUARE)
- La casilla `a1` es oscura.
- Las piezas se imprimen con su símbolo Unicode según código de entrada.
- Cada fila lleva 8 símbolos separados por **un espacio**.

# A. Dibujando tableros — Resumen

## ¿Qué se pide?

Dado un listado de piezas con coordenadas, imprimir el estado final de un tablero de ajedrez de $8 \times 8$.

- Casillas vacías en Unicode:
  - clara: `\u25FB` (WHITE SQUARE)
  - oscura: `\u25FC` (BLACK SQUARE)
- La casilla `a1` es oscura.
- Las piezas se imprimen con su símbolo Unicode según código de entrada.
- Cada fila lleva 8 símbolos separados por **un espacio**.

## Entrada

- Hasta $T = 10^5$ casos.
- Por caso: $0 \leq P \leq 32$ piezas con formato `C X`.

# A. Dibujando tableros — Estrategia

**Idea**

Construir una matriz `board[8][8]` e ir colocando piezas por coordenada.

# A. Dibujando tableros — Estrategia

**Idea**

Construir una matriz `board[8][8]` e ir colocando piezas por coordenada.

- Convertir `X=columna+fila` a índices:

$$columna = X[0] -' a', \quad fila = X[1] -' 1'$$

- Guardar la pieza en `board[fila][columna]`.
- Imprimir filas de 7 a 0 (de 8 a 1) y columnas de 0 a 7 (de a a h).
- Si no hay pieza, calcular el color por paridad.
- Restamos `'a'` y `'1'` para convertir carácter a índice del array: como en ASCII `'b'-'a'=1` y `'2'-'1'=1`, obtenemos sus posiciones numéricas en el array.

# A. Dibujando tableros — Estrategia

## Idea

Construir una matriz `board[8][8]` e ir colocando piezas por coordenada.

- Convertir `X=columna+fila` a índices:

$$\mathit{columna} = X[0] -' a', \quad \mathit{fila} = X[1] -' 1'$$

- Guardar la pieza en `board[fila][columna]`.
- Imprimir filas de 7 a 0 (de 8 a 1) y columnas de 0 a 7 (de a a h).
- Si no hay pieza, calcular el color por paridad.
- Restamos `'a'` y `'1'` para convertir carácter a índice del array: como en ASCII `'b'-'a'=1` y `'2'-'1'=1`, obtenemos sus posiciones numéricas en el array.

## Paridad de color

Como `a1` es oscura, una casilla es oscura si

$$(\mathit{columna} + \mathit{fila}) \text{ mód } 2 = 0$$

# A. Dibujando tableros — Complejidad

- Colocar piezas: $\mathcal{O}(P)$
- Imprimir tablero: $\mathcal{O}(64)$

# A. Dibujando tableros — Complejidad

- Colocar piezas: $\mathcal{O}(P)$
- Imprimir tablero: $\mathcal{O}(64)$

**Total por caso**

$$\mathcal{O}(P + 64) = \mathcal{O}(P), \quad P \leq 32$$

# A. Dibujando tableros — Complejidad

- Colocar piezas: $\mathcal{O}(P)$
- Imprimir tablero: $\mathcal{O}(64)$

## Total por caso

$$\mathcal{O}(P + 64) = \mathcal{O}(P), \quad P \leq 32$$

## Memoria

$$\mathcal{O}(64)$$

(tablero fijo de 8x8)

# A. Dibujando tableros — Errores comunes

- **Orientación invertida**: imprimir de fila 1 a 8 en vez de 8 a 1.

# A. Dibujando tableros — Errores comunes

- **Orientación invertida**: imprimir de fila 1 a 8 en vez de 8 a 1.
- **Paridad mal aplicada**: olvidar que `a1` es oscura.

# A. Dibujando tableros — Errores comunes

- **Orientación invertida**: imprimir de fila 1 a 8 en vez de 8 a 1.
- **Paridad mal aplicada**: olvidar que `a1` es oscura.
- **Separador incorrecto**: usar tabulador en vez de espacio.

# A. Dibujando tableros — Errores comunes

- **Orientación invertida**: imprimir de fila 1 a 8 en vez de 8 a 1.
- **Paridad mal aplicada**: olvidar que `a1` es oscura.
- **Separador incorrecto**: usar tabulador en vez de espacio.
- **Espacio final**: dejar un espacio al final de línea.

# A. Dibujando tableros — Errores comunes

- **Orientación invertida**: imprimir de fila 1 a 8 en vez de 8 a 1.
- **Paridad mal aplicada**: olvidar que `a1` es oscura.
- **Separador incorrecto**: usar tabulador en vez de espacio.
- **Espacio final**: dejar un espacio al final de línea.
- **Mapeo Unicode**: confundir letras de piezas (`R,D,T,A,C,P` y minúsculas).

# E. El Número Misterioso

| Envíos | Válidos | % éxito |
|---|---|---|
| 57 | 55 | 96 % |

# E. El Número Misterioso — Resumen

## ¿Qué se pide?

Calcular en cuantas iteraciones se obtiene el número misterioso 6174 a partir de un número de cuatro cifras, en el que todas las cifras no pueden ser iguales. Por ejemplo, para el número 7283:

- Iteración 1: máximo = 8732, mínimo = 2378. diferencia = 8732 − 2378 = 6354
- Iteración 2: máximo = 6543, mínimo = 3456, diferencia = 3087
- Iteración 3: máximo = 8730, mínimo = 378, diferencia = 8352
- Iteración 4: máximo = 8532, mínimo = 2358, diferencia = 6174.

En este caso se han necesitado 4 iteraciones

## Restricciones

Solo hay una única restricción

- Las cuatro cifras del número no pueden ser iguales, de esta forma los únicos números no válidos serían 1111, 2222, 3333, 4444, 5555, 6666, 7777, 8888 y 9999.

# E. El Número Misterioso — La clave y la solución

## ¿La clave está en cómo obtener el máximo y el mínimo posible permutando las cifras de un número de 4 cifras?

- Descomponer el número en sus cuatro cifras y almacenarlas en un vector.

```
Sean n el número dado y v el vector de cuatro enteros.
resto = n
para i de 3 a 1 inc -1 hacer
    v[i] = resto div 10^i
    resto = resto mod 10^i
finpara
v[0] = resto
```

- Obtener el máximo y el mínimo.

```
Ordenar v descendentemente
maximo = 0
minimo = 0
para i de 0 a 3 hacer
    maximo = maximo + v[i] * 10^(3-i)
    minimo = minimo + v[i] * 10^i
finpara
```

- J. AByronOS

| Envíos | Válidos | % éxito |
|---|---|---|
| 38 | 22 | 58 % |

# J. AByronOS — Resumen

## ¿Qué se pide?

Simular el comportamiento de un **planificador de procesos**:

- Se recibe una secuencia de $N$ enteros.
- Los valores **positivos** representan procesos que llegan.
- Los valores **negativos** representan cancelaciones de procesos previos.

## Restricciones

- $10 \leq N \leq 10^5$.
- Cada valor es un entero de 32 bits.
- Cancelaciones consecutivas pueden acumularse.
- El algoritmo debe ser eficiente en tiempo y memoria.

# J. AByronOS — Resumen

## Fase 1

El comportamiento natural del sistema se modela con una **pila**:

- Los procesos entran al final.
- Las cancelaciones eliminan desde el final. Si el número de cancelaciones supera el tamaño de la pila: la pila queda vacía.

## Fase 2

Una vez resueltas las cancelaciones, y si quedan procesos, la estructura se interpreta como una **lista de procesos**:

- Se recorre secuencialmente en bloques consecutivos de 4 elementos.
- En cada bloque:
  - Primero los procesos **pares**.
  - Luego los procesos **impares**.

# J. AByronOS — Fase 1

## Registro y Cancelación

Recorremos la entrada una sola vez:

- Si $x \geq 0$: se añade $x$ a la pila.
- Si $x < 0$: se eliminan $|x|$ procesos desde el final.
- Se puede acumular cancelaciones consecutivas para optimizar las operaciones de cancelación que se hacen en la pila en una única operación.

# J. AByronOS — Fase 1

## Registro y Cancelación

Recorremos la entrada una sola vez:

- Si $x \geq 0$: se añade $x$ a la pila.
- Si $x < 0$: se eliminan $|x|$ procesos desde el final.
- Se puede acumular cancelaciones consecutivas para optimizar las operaciones de cancelación que se hacen en la pila en una única operación.

## Detalle importante

Si el número de cancelaciones supera el tamaño de la pila: **la pila queda vacía** y se continúa con el siguiente elemento de la entrada.

# J. AByronOS — Fase 2

## Reorganización final

Si la pila no está vacía, su contenido se trata como una **lista de procesos**:

- La lista se divide en **sublistas contiguas** de tamaño 4.
- Cada sublista se procesa de forma independiente.
- Para cada sublista se colocan:
  - primero los procesos **pares**,
  - y a continuación los procesos **impares**.
- El resultado de cada sublista se concatena en la salida final, preservando el orden entre sublistas.

# J. AByronOS — Fase 2

## Reorganización final

Si la pila no está vacía, su contenido se trata como una **lista de procesos**:

- La lista se divide en **sublistas contiguas** de tamaño 4.
- Cada sublista se procesa de forma independiente.
- Para cada sublista se colocan:
  - primero los procesos **pares**,
  - y a continuación los procesos **impares**.
- El resultado de cada sublista se concatena en la salida final, preservando el orden entre sublistas.

## Caso especial

Si no ha quedado ningún proceso de la fase 1: `NOPROCESO`

# J. AByronOS — Ejemplo

**Entrada**

```
5 2 9 12 7 8 -1 -2 4 11 6
```

# J. AByronOS — Ejemplo

## Entrada

```
5 2 9 12 7 8 -1 -2 4 11 6
```

## Tras Fase 1

```
[5, 2, 9, 4, 11, 6]
```

# J. AByronOS — Ejemplo

## Entrada

```
5 2 9 12 7 8 -1 -2 4 11 6
```

## Tras Fase 1

```
[5, 2, 9, 4, 11, 6]
```

## Tras Fase 2

```
2 4 5 9 6 11
```

# K. Balanceando la Banda

| Envíos | Válidos | % éxito |
|---|---|---|
| 8 | 4 | 50 % |

# K. Balanceando la Banda — Resumen

## ¿Qué se pide?

Organizar una banda de hermandad de Warcraft:

- Hay $M$ jugadores, cada uno domina **dos roles**.
- Límite por rol: máximo $\lfloor N/K \rfloor$ jugadores.

Determinar si existe una distribución válida para los $M$ jugadores.

# K. Balanceando la Banda — Resumen

## ¿Qué se pide?

Organizar una banda de hermandad de Warcraft:

- Hay $M$ jugadores, cada uno domina **dos roles**.
- Límite por rol: máximo $\lfloor N/K \rfloor$ jugadores.

Determinar si existe una distribución válida para los $M$ jugadores.

## Restricciones

- $1 \leq M \leq 100$ jugadores.
- $1 \leq K \leq 20$ roles.
- $M \leq N \leq 200$ (N es múltiplo de K).

# K. Balanceando la Banda — Idea Clave

## ¿Por qué es difícil?

No es una asignación independiente:

- La decisión de un jugador $A$ de tomar el rol $R_1$ puede "robarle" el sitio a un jugador $B$ que solo sabe usar $R_1$ y $R_2$.
- Necesitamos un mecanismo que permita 'rectificar" asignaciones previas si es necesario.

# K. Balanceando la Banda — Idea Clave

## ¿Por qué es difícil?

No es una asignación independiente:

- La decisión de un jugador $A$ de tomar el rol $R_1$ puede "robarle" el sitio a un jugador $B$ que solo sabe usar $R_1$ y $R_2$.
- Necesitamos un mecanismo que permita 'rectificar" asignaciones previas si es necesario.

## La Solución

Estamos ante un problema de **Emparejamiento Bipartito con capacidades** o, de forma más general, **Flujo Máximo**.

# K. Balanceando la Banda — Modelado de la Red

Construimos una red de flujo con $M + K + 2$ nodos:

# K. Balanceando la Banda — Modelado de la Red

Construimos una red de flujo con $M + K + 2$ nodos:

- **Fuente** $\rightarrow$ **Jugadores**: Capacidad $1$.
- **Jugadores** $\rightarrow$ **Roles**: Aristas de capacidad $1$ hacia sus $2$ especialidades.
- **Roles** $\rightarrow$ **Sumidero**: Capacidad $L = N/K$.

# K. Balanceando la Banda — Modelado de la Red

Construimos una red de flujo con $M + K + 2$ nodos:

- **Fuente** $\rightarrow$ **Jugadores**: Capacidad $1$.
- **Jugadores** $\rightarrow$ **Roles**: Aristas de capacidad $1$ hacia sus $2$ especialidades.
- **Roles** $\rightarrow$ **Sumidero**: Capacidad $L = N/K$.

## Criterio de éxito

Si el **flujo máximo** es exactamente $M$, todos tienen sitio $\Rightarrow$ `YES`.

# K. Balanceando la Banda — Modelado de la Red

Construimos una red de flujo con $M + K + 2$ nodos:

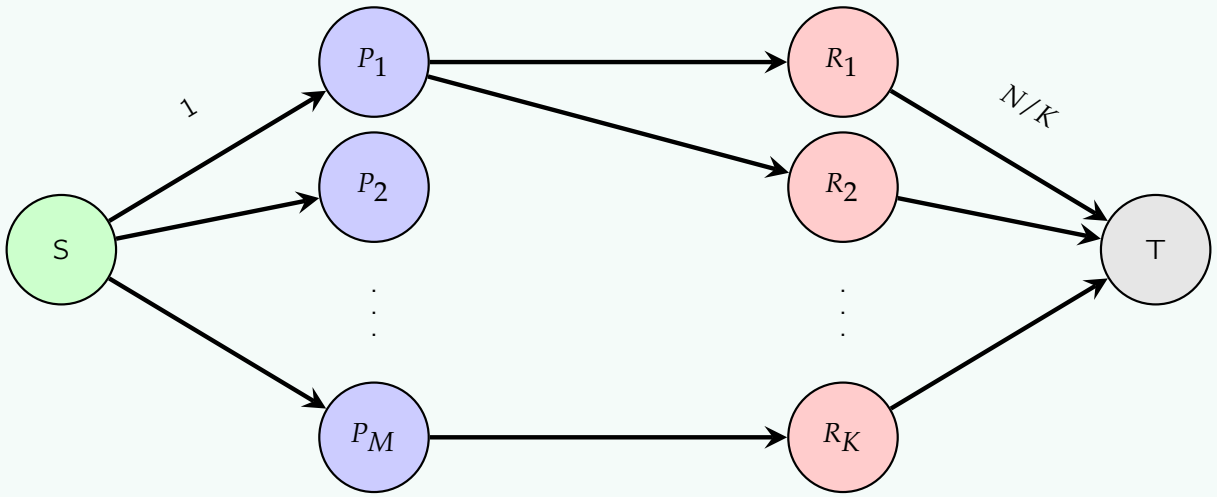
- **Fuente** $\rightarrow$ **Jugadores**: Capacidad $1$.
- **Jugadores** $\rightarrow$ **Roles**: Aristas de capacidad $1$ hacia sus 2 especialidades.
- **Roles** $\rightarrow$ **Sumidero**: Capacidad $L = N/K$.

## Criterio de éxito

Si el **flujo máximo** es exactamente $M$, todos tienen sitio $\Rightarrow$ `YES`.

Complejidad: $\mathcal{O}(V \cdot E^2)$ o $\mathcal{O}(E\sqrt{V})$. Muy eficiente para $M = 100, K = 20$.

# K. Balanceando la Banda — Modelado Visual



# K. Balanceando la Banda — Alternativa: Emparejamiento Máximo

## Otra perspectiva

El problema también podría haberse tratado como un problema de **emparejamiento máximo** en un grafo bipartito.

# K. Balanceando la Banda — Alternativa: Emparejamiento Máximo

## Otra perspectiva

El problema también podría haberse tratado como un problema de **emparejamiento máximo** en un grafo bipartito.

## Algoritmo de Kuhn

El algoritmo de Kuhn es un método eficiente de tiempo polinomial utilizado para encontrar el emparejamiento máximo en un grafo bipartito.

# K. Balanceando la Banda — Alternativa: Emparejamiento Máximo

## Otra perspectiva

El problema también podría haberse tratado como un problema de **emparejamiento máximo** en un grafo bipartito.

## Algoritmo de Kuhn

El algoritmo de Kuhn es un método eficiente de tiempo polinomial utilizado para encontrar el emparejamiento máximo en un grafo bipartito.

- **Idea**: Expandir cada rol en $L = N/K$ "slots" individuales y buscar el emparejamiento de los $M$ jugadores.
- **Ventaja**: Muy sencillo de implementar con un simple DFS.

# K. Balanceando la Banda — Fallos Comunes

- **Greedy simple**: asignar el primer rol libre que vemos. No permite corregir errores pasados.

# K. Balanceando la Banda — Fallos Comunes

- **Greedy simple**: asignar el primer rol libre que vemos. No permite corregir errores pasados.
- **Backtracking**: probar todas las combinaciones es $\mathcal{O}(2^M)$. Con $M = 100$, es impracticable.

# K. Balanceando la Banda — Fallos Comunes

- **Greedy simple**: asignar el primer rol libre que vemos. No permite corregir errores pasados.
- **Backtracking**: probar todas las combinaciones es $\mathcal{O}(2^M)$. Con $M = 100$, es impracticable.
- **Cálculo del límite**: usar $N$ directamente en lugar de $N/K$.

# K. Balanceando la Banda — Fallos Comunes

- **Greedy simple**: asignar el primer rol libre que vemos. No permite corregir errores pasados.
- **Backtracking**: probar todas las combinaciones es $\mathcal{O}(2^M)$. Con $M = 100$, es impracticable.
- **Cálculo del límite**: usar $N$ directamente en lugar de $N/K$.
- **Manejo de nombres**: no resetear el diccionario/mapa de nombres de roles entre casos de prueba.

# H. Acceso a Laboratorios

| Envíos | Válidos | % éxito |
|---|---|---|
| 59 | 54 | 92 % |

# H. Acceso a Laboratorios — Resumen

## ¿Qué se pide?

Tenemos un laboratorio que admite un aforo de $C$ personas. Se controla cuando persona entra (`E`) o sale (`S`) del laboratorio. Se pide, dado un registro de las entradas y las salidas ordenado según se producen indicar si es `VALIDO` o `INVALIDO`.

# H. Acceso a Laboratorios — Resumen

## ¿Qué se pide?

Tenemos un laboratorio que admite un aforo de $C$ personas. Se controla cuando persona entra (`E`) o sale (`S`) del laboratorio. Se pide, dado un registro de las entradas y las salidas ordenado según se producen indicar si es `VALIDO` o `INVALIDO`.

## ¿Cuándo es `VALIDO`?

Cuando no es `INVALIDO`...

# H. Acceso a Laboratorios — Resumen

## ¿Qué se pide?

Tenemos un laboratorio que admite un aforo de $C$ personas. Se controla cuando persona entra (`E`) o sale (`S`) del laboratorio. Se pide, dado un registro de las entradas y las salidas ordenado según se producen indicar si es `VALIDO` o `INVALIDO`.

## ¿Cuándo es `VALIDO`?

Cuando no es `INVALIDO`...

## ¿Cuándo es `INVALIDO`?

- Se observa una salida estando la sala vacía.
- Se sobrepasa el aforo.
- Al finalizar el registro, queda alguien en la sala.

# H. Acceso a Laboratorios — Solución

## Idea básica

Tenemos una variable que cuenta cuántas personas hay en el laboratorio en cada momento: `personas`.
Se va leyendo el registro modificando ese contador:

- Si se lee una entrada (`E`) se incrementa el contador (`personas++`)
- Si se lee una salida (`S`) se decrementa el contador (`personas--`)

# H. Acceso a Laboratorios — Solución

## Idea básica

Tenemos una variable que cuenta cuántas personas hay en el laboratorio en cada momento: `personas`.
Se va leyendo el registro modificando ese contador:

- Si se lee una entrada (`E`) se incrementa el contador (`personas++`)
- Si se lee una salida (`S`) se decrementa el contador (`personas--`)

## Comprobado la validez

Inicialmente se considera válido salvo que:

- Hay una salida con la sala vacía: (`personas < 0`) en cualquier momento.
- Se sobrepasa el aforo: (`personas > C`) en cualquier momento.
- Queda alguien en la sala al finalizar: (`personas != 0`) al finalizar.

## B. Encriptación matricial

| Envíos | Válidos | % éxito |
|---|---|---|
| 7 | 7 | 100 % |

# B. Encriptación matricial — Resumen

## ¿Qué se pide?

Dada una matriz de entrada de tamaño $12 \times 3$ con dígitos decimales entre $0$ y $9$, hay que aplicar un proceso de encriptación en tres pasos:

1. Convertir cada número a binario de 4 bits para construir una matriz binaria de tamaño $12 \times 12$.
2. Dividir esa matriz en 16 submatrices de tamaño $3 \times 3$ y, en cada una, rotar la periferia según el número de unos y el valor del elemento central.
3. Transformar la matriz binaria final en un vector de 12 números decimales, interpretando cada columna como un número binario de 12 bits.

El objetivo final es imprimir ese vector resultante.

# B. Encriptación matricial — Resumen

## Paso 1

Se convierte la matriz original $12 \times 3$ en una matriz binaria $12 \times 12$.
Cada número decimal se representa usando 4 bits en binario. Después, los 3 números de cada fila se concatenan, formando así una fila de 12 bits.

**Ejemplo:** si una fila de la matriz de entrada es

$$3 \quad 8 \quad 1$$

entonces su conversión binaria es

$$3 \to 0011, \qquad 8 \to 1000, \qquad 1 \to 0001$$

y, al concatenarlos, se obtiene la fila binaria

$$001110000001$$

# B. Encriptación matricial — Resumen

## Paso 2

La matriz binaria $12 \times 12$ resultante se divide en 16 bloques $3 \times 3$. En cada bloque se toma la periferia (los 8 elementos exteriores) y se cuenta cuántos unos hay; a ese número lo llamamos $N$.

- Si el centro es $0$, la periferia se rota $N$ posiciones a la izquierda.
- Si el centro es $1$, la periferia se rota $N$ posiciones a la derecha.

**Ejemplo:** supongamos el bloque

$$\begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 0 & 1 & 1 \end{pmatrix}$$

Su periferia tiene 3 unos, luego $N = 3$. Como el elemento central vale $1$, la periferia se rota 3 posiciones hacia la derecha. El bloque resultante es

$$\begin{pmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 0 & 1 & 0 \end{pmatrix}$$

# B. Encriptación matricial — Resultado de las rotaciones

## Paso 2 - Matriz binaria resultante

Tras procesar las 16 submatrices de tamaño $3 \times 3$ y aplicar en cada una la rotación correspondiente de su periferia, se obtiene la siguiente matriz binaria final:

$$\left[\begin{array}{ccc|ccc|ccc|ccc}
1 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 0 \\
1 & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 \\
0 & 1 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 0 \\
\hline
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 \\
0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 \\
0 & 1 & 1 & 0 & 1 & 1 & 0 & 1 & 0 & 0 & 1 & 0 \\
\hline
0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \\
1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \\
0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\
\hline
0 & 1 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 \\
1 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0
\end{array}\right]$$

# B. Encriptación matricial — Resumen

## Paso 3

La matriz binaria final se transforma en un vector de 12 números decimales. Para ello, cada columna se interpreta como un número binario de 12 bits, leído de arriba hacia abajo, y después se convierte a decimal.

**Ejemplo:** la primera columna de la matriz final del enunciado es

$$1\ 1\ 0\ 0\ 0\ 0\ 0\ 1\ 0\ 0\ 0\ 1$$

Por tanto, se interpreta como el número binario

$$110000010001_2$$

y su valor decimal es

$$2^{11} + 2^{10} + 2^4 + 2^0 = 2048 + 1024 + 16 + 1 = 3089.$$

Repitiendo este proceso con las 12 columnas, se obtiene el vector final de salida.

# B. Encriptación matricial — Ejemplo de entrada

## Ejemplo de entrada

La matriz de entrada del ejemplo es una matriz de tamaño $12 \times 3$:

$$\begin{matrix} 3 & 8 & 1 \\ 5 & 2 & 9 \\ 7 & 4 & 3 \\ 1 & 6 & 2 \\ 9 & 0 & 4 \\ 8 & 3 & 6 \\ 2 & 5 & 7 \\ 0 & 9 & 1 \\ 6 & 8 & 2 \\ 4 & 7 & 5 \\ 1 & 3 & 8 \\ 7 & 6 & 0 \end{matrix}$$

Cada elemento de esta matriz se convierte a binario usando 4 bits.

# B. Encriptación matricial — Matriz binaria resultante

## Matriz binaria inicial

Tras convertir cada número decimal a binario de 4 bits, se obtiene la siguiente matriz binaria de tamaño $12 \times 12$:

$$\left[\begin{array}{ccc|ccc|ccc|ccc}
0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\
0 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 \\
0 & 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 1 \\
\hline
0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 1 & 0 \\
1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\
1 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 0 \\
\hline
0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 1 & 1 \\
0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \\
0 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\
\hline
0 & 1 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 1 & 0 & 1 \\
0 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 \\
0 & 1 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0
\end{array}\right]$$

Esta matriz se divide en 16 submatrices $3 \times 3$ para aplicar las rotaciones.

# B. Encriptación matricial — Matriz binaria rotada

## Matriz binaria tras las rotaciones

Después de procesar las 16 submatrices $3 \times 3$ y rotar la periferia de cada una según el algoritmo, se obtiene la matriz binaria final:

$$\left[\begin{array}{ccc|ccc|ccc|ccc}
1 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 0 \\
1 & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 \\
0 & 1 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 0 \\
\hline
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 \\
0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 \\
0 & 1 & 1 & 0 & 1 & 1 & 0 & 1 & 0 & 0 & 1 & 0 \\
\hline
0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \\
1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \\
0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\
\hline
0 & 1 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 \\
1 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0
\end{array}\right]$$

# B. Encriptación matricial — Resultado final

## Salida

Para obtener el resultado final, cada columna de la matriz binaria rotada se interpreta como un número binario de 12 bits leído de arriba hacia abajo. Así, el vector de salida del ejemplo es:

$$3089 \quad 1636 \quad 100 \quad 1173 \quad 2128 \quad 3662 \quad 5 \quad 2643 \quad 419 \quad 3629 \quad 2376 \quad 408$$

## D. Tunelando el futuro

| Envíos | Válidos | % éxito |
|---|---|---|
| 15 | 12 | 80 % |

# D. Tunelando el futuro — Resumen

## ¿Qué se pide?

Encontrar una forma de conectar ciudades-búnker mediante túneles con el mínimo coste, teniendo en cuenta que:

- Solo hay un equipo que tiene que hacer todos los túneles.
- El material que hay entre dos ciudades afecta al ritmo de avance de la tuneladora.
- Cada día de trabajo del equipo tiene un coste fijo de un millón de euros.

# D. Tunelando el futuro — Resumen

## ¿Qué se pide?

Encontrar una forma de conectar ciudades-búnker mediante túneles con el mínimo coste, teniendo en cuenta que:

- Solo hay un equipo que tiene que hacer todos los túneles.
- El material que hay entre dos ciudades afecta al ritmo de avance de la tuneladora.
- Cada día de trabajo del equipo tiene un coste fijo de un millón de euros.

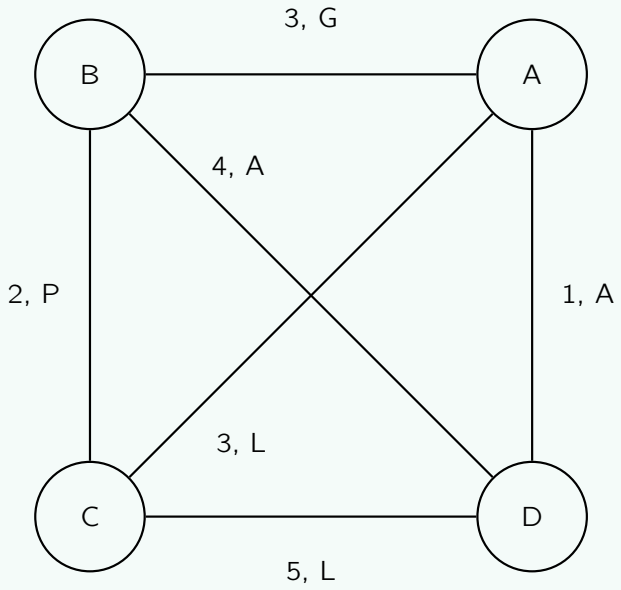
## Restricciones

El número de ciudades-búnker $N$ a conectar es menor o igual que 1000.

# D. Tunelando el futuro — Idea clave

## ¿Cómo se resuelve?

- Se modela como un problema de grafos.
- Cada ciudad es un nodo de un grafo completo.
- Los ejes representan posibles túneles a excavar.
- Para conectar todas las ciudades-búnker basta con encontrar un árbol recubridor mínimo.
- Es importante establecer correctamente los pesos de los ejes.



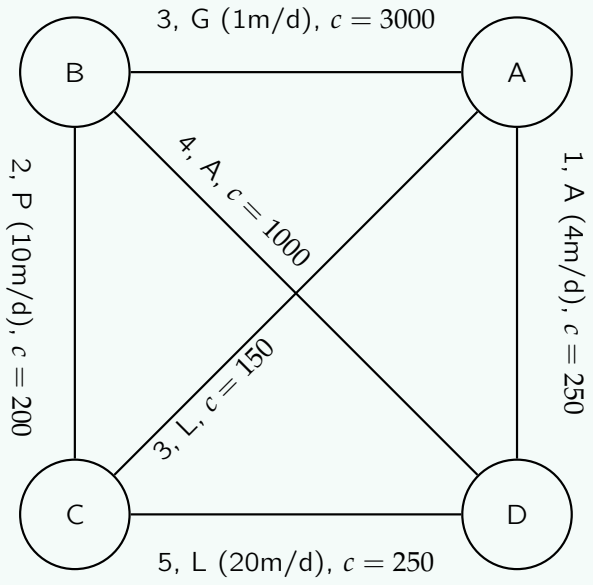
# D. Tunelando el futuro — Idea clave

## Pesos de los ejes

- El peso debe ser el coste de excavar el túnel.
- El coste de excavar un túnel es proporcional al tiempo de trabajo del equipo.
- El tiempo requerido para avanzar en un túnel es propocional a la distancia e inversamente proporcional a la velocidad de avance.
- La velocidad de avance la determina el material.
- Cada material tiene asociada una velocidad de avance

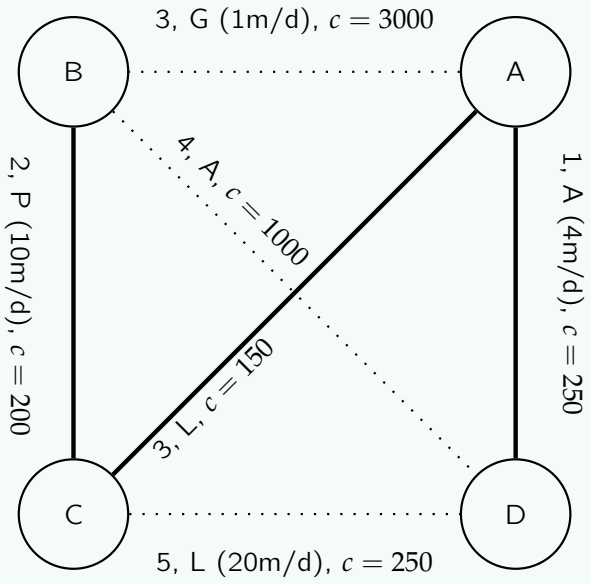
El coste asociado a una arista (en millones de euros) se calcula como: $c = \frac{1000 \cdot d}{v}$, donde $d$ es la distancia en kilómetros y $v$ es la velocidad de de avance en metros por día, que viene determinado por el material.

# D. Tunelando el futuro — Ejemplo



# D. Tunelando el futuro — Solución

Ahora solo queda aplicar el algoritmo de Prim o Kruskal para encontrar un árbol recubridor mínimo. Por ejemplo, si aplicamos el algoritmo de Kruskal podemos ordenar las aristas de forma creciente en el coste. Se puede representar el conjunto de árboles en cada momento con una estructura de tipo UnionFind. Se itera sobre toda las aristas: si una arista une nodos del mismo árbol se descarta, en caso contrario se añade su coste al coste global y se unen los árboles que conecta.




Coste total: 600 millones de euros.

# D. Tunelando el futuro — Fallos comunes

- **Identificar el problema:** no darse cuenta de que se trata de un problema de búsqueda de árbol recubridor mínimo, en lugar de un problema de caminos más cortos.

# D. Tunelando el futuro — Fallos comunes

- **Identificar el problema:** no darse cuenta de que se trata de un problema de búsqueda de árbol recubridor mínimo, en lugar de un problema de caminos más cortos.
- **Cálculo de coste:** calcular incorrectamente el coste de las aristas del grafo.

# D. Tunelando el futuro — Fallos comunes

- **Identificar el problema:** no darse cuenta de que se trata de un problema de búsqueda de árbol recubridor mínimo, en lugar de un problema de caminos más cortos.
- **Cálculo de coste:** calcular incorrectamente el coste de las aristas del grafo.
- **Implementación del algoritmo:** cometer errores en la implementación del algoritmo, como no ordenar las aristas de forma creciente en coste.

## L. Descubriendo Genómicos

| Envíos | Válidos | % éxito |
|---|---|---|
| 0 | 0 | – |

# L. Descubriendo Genómicos — Resumen

## ¿Qué se pide?

Dada una cadena de ADN y una lista de motivos, encontrar la longitud de la **subcadena contigua más corta** que contenga al menos $k$ motivos **diferentes** de la lista.

# L. Descubriendo Genómicos — Resumen

## ¿Qué se pide?

Dada una cadena de ADN y una lista de motivos, encontrar la longitud de la **subcadena contigua más corta** que contenga al menos $k$ motivos **diferentes** de la lista.

## Observación clave

Un enfoque exhaustivo, y poco eficiente, analiza todas las subcadenas posibles, comprobando en cada caso la presencia de los distintos motivos.

# L. Descubriendo Genómicos — Resumen

## ¿Qué se pide?

Dada una cadena de ADN y una lista de motivos, encontrar la longitud de la **subcadena contigua más corta** que contenga al menos $k$ motivos **diferentes** de la lista.

## Observación clave

Un enfoque exhaustivo, y poco eficiente, analiza todas las subcadenas posibles, comprobando en cada caso la presencia de los distintos motivos.

## Problema de la solución exhaustiva

Si la cadena es muy larga, generar todas las subcadenas y buscar en cada una todos los motivos resulta demasiado costoso.

# L. Descubriendo Genómicos — Resumen

## ¿Qué se pide?

Dada una cadena de ADN y una lista de motivos, encontrar la longitud de la **subcadena contigua más corta** que contenga al menos $k$ motivos **diferentes** de la lista.

## Observación clave

Un enfoque exhaustivo, y poco eficiente, analiza todas las subcadenas posibles, comprobando en cada caso la presencia de los distintos motivos.

## Problema de la solución exhaustiva

Si la cadena es muy larga, generar todas las subcadenas y buscar en cada una todos los motivos resulta demasiado costoso.

## Idea principal:

en lugar de examinar todas las subcadenas, conviene trabajar directamente con las **ocurrencias reales** de los motivos dentro del ADN.

# L. Descubriendo Genómicos — Solución optimizada

**Paso 1:** localizar todas las ocurrencias de cada motivo en la cadena de ADN.

# L. Descubriendo Genómicos — Solución optimizada

**Paso 1:** localizar todas las ocurrencias de cada motivo en la cadena de ADN. Cada ocurrencia puede representarse como:

$$(\text{fin},\ \text{inicio},\ \text{idMotivo})$$

donde:

- `inicio` = posición donde empieza el motivo,
- `fin` = posición donde termina,
- `idMotivo` = motivo al que pertenece.

# L. Descubriendo Genómicos — Solución optimizada

**Paso 1:** localizar todas las ocurrencias de cada motivo en la cadena de ADN. Cada ocurrencia puede representarse como:

$$(\text{fin},\ \text{inicio},\ \text{idMotivo})$$

donde:

- `inicio` = posición donde empieza el motivo,
- `fin` = posición donde termina,
- `idMotivo` = motivo al que pertenece.

**Paso 2:** ordenar todas las ocurrencias por su posición final.

# L. Descubriendo Genómicos — Solución optimizada

**Paso 1:** localizar todas las ocurrencias de cada motivo en la cadena de ADN. Cada ocurrencia puede representarse como:

$$(\text{fin},\ \text{inicio},\ \text{idMotivo})$$

donde:

- `inicio` = posición donde empieza el motivo,
- `fin` = posición donde termina,
- `idMotivo` = motivo al que pertenece.

**Paso 2:** ordenar todas las ocurrencias por su posición final.
**Paso 3:** recorrer esas ocurrencias de izquierda a derecha manteniendo, para cada motivo, la ocurrencia válida más a la derecha vista hasta el momento.

# L. Descubriendo Genómicos — Solución optimizada

**Paso 1:** localizar todas las ocurrencias de cada motivo en la cadena de ADN. Cada ocurrencia puede representarse como:

$$(\text{fin},\ \text{inicio},\ \text{idMotivo})$$

donde:

- `inicio` = posición donde empieza el motivo,
- `fin` = posición donde termina,
- `idMotivo` = motivo al que pertenece.

**Paso 2:** ordenar todas las ocurrencias por su posición final.
**Paso 3:** recorrer esas ocurrencias de izquierda a derecha manteniendo, para cada motivo, la ocurrencia válida más a la derecha vista hasta el momento.
**Paso 4:** cuando ya hay al menos $k$ motivos activos, tomamos los $k$ mayores valores de inicio. Si el menor de ellos es $L$ y la ocurrencia actual termina en $R$, entonces:

$$\text{longitud de la ventana} = R - L + 1$$

# L. Descubriendo Genómicos — Solución optimizada

**Paso 1:** localizar todas las ocurrencias de cada motivo en la cadena de ADN. Cada ocurrencia puede representarse como:

$$(\text{fin},\ \text{inicio},\ \text{idMotivo})$$

donde:

- `inicio` = posición donde empieza el motivo,
- `fin` = posición donde termina,
- `idMotivo` = motivo al que pertenece.

**Paso 2:** ordenar todas las ocurrencias por su posición final.
**Paso 3:** recorrer esas ocurrencias de izquierda a derecha manteniendo, para cada motivo, la ocurrencia válida más a la derecha vista hasta el momento.
**Paso 4:** cuando ya hay al menos $k$ motivos activos, tomamos los $k$ mayores valores de inicio. Si el menor de ellos es $L$ y la ocurrencia actual termina en $R$, entonces:

$$\text{longitud de la ventana} = R - L + 1$$

**Ventaja:** evitamos generar todas las subcadenas y solo trabajamos con posiciones de ocurrencias.

# L. Descubriendo Genómicos — Justificación y complejidad

## ¿Por qué funciona?

Una ventana $[L, R]$ contiene un motivo si existe una ocurrencia completa de ese motivo cuyo inicio sea $\geq L$ y cuyo fin sea $\leq R$.

# L. Descubriendo Genómicos — Justificación y complejidad

## ¿Por qué funciona?

Una ventana $[L, R]$ contiene un motivo si existe una ocurrencia completa de ese motivo cuyo inicio sea $\geq L$ y cuyo fin sea $\leq R$.
Para un extremo derecho fijo $R$, interesa conservar para cada motivo la ocurrencia con **inicio más a la derecha**, porque permite acortar al máximo la ventana.

# L. Descubriendo Genómicos — Justificación y complejidad

## ¿Por qué funciona?

Una ventana $[L, R]$ contiene un motivo si existe una ocurrencia completa de ese motivo cuyo inicio sea $\geq L$ y cuyo fin sea $\leq R$.
Para un extremo derecho fijo $R$, interesa conservar para cada motivo la ocurrencia con **inicio más a la derecha**, porque permite acortar al máximo la ventana.

## Coste

- Búsqueda de ocurrencias de los motivos en el ADN.
- Ordenación de las ocurrencias encontradas.
- Recorrido lineal de dichas ocurrencias manteniendo la mejor ventana.

# L. Descubriendo Genómicos — Justificación y complejidad

## ¿Por qué funciona?

Una ventana $[L, R]$ contiene un motivo si existe una ocurrencia completa de ese motivo cuyo inicio sea $\geq L$ y cuyo fin sea $\leq R$.
Para un extremo derecho fijo $R$, interesa conservar para cada motivo la ocurrencia con **inicio más a la derecha**, porque permite acortar al máximo la ventana.

## Coste

- Búsqueda de ocurrencias de los motivos en el ADN.
- Ordenación de las ocurrencias encontradas.
- Recorrido lineal de dichas ocurrencias manteniendo la mejor ventana.

$$\text{Complejidad} \approx \mathcal{O}(T \log T)$$

donde $T$ es el número total de ocurrencias encontradas (asumiendo $m$ pequeño, número de motivos).

# L. Descubriendo Genómicos — Justificación y complejidad

## ¿Por qué funciona?

Una ventana $[L, R]$ contiene un motivo si existe una ocurrencia completa de ese motivo cuyo inicio sea $\geq L$ y cuyo fin sea $\leq R$.
Para un extremo derecho fijo $R$, interesa conservar para cada motivo la ocurrencia con **inicio más a la derecha**, porque permite acortar al máximo la ventana.

## Coste

- Búsqueda de ocurrencias de los motivos en el ADN.
- Ordenación de las ocurrencias encontradas.
- Recorrido lineal de dichas ocurrencias manteniendo la mejor ventana.

$$\text{Complejidad} \approx \mathcal{O}(T \log T)$$

donde $T$ es el número total de ocurrencias encontradas (asumiendo $m$ pequeño, número de motivos).

Frente a la solución exhaustiva, la mejora es muy importante porque ya no se enumeran todas las subcadenas posibles del ADN.

# L. Descubriendo Genómicos — Ejemplo

**Entrada:**

- ADN: `ACGTACGT`
- Motivos: `ACG`, `CGT`, `TAC`
- $k = 2$

# L. Descubriendo Genómicos — Ejemplo

**Entrada:**

- ADN: `ACGTACGT`
- Motivos: `ACG`, `CGT`, `TAC`
- $k = 2$

**Ocurrencias:**

- `ACG`: [0,2], [4,6]
- `CGT`: [1,3], [5,7]
- `TAC`: [3,5]

# L. Descubriendo Genómicos — Ejemplo

**Entrada:**

- ADN: `ACGTACGT`
- Motivos: `ACG`, `CGT`, `TAC`
- $k = 2$

**Ocurrencias:**

- `ACG`: [0,2], [4,6]
- `CGT`: [1,3], [5,7]
- `TAC`: [3,5]

**Ventana candidata:**

$$[0, 3] = \texttt{ACGT}$$

# L. Descubriendo Genómicos — Ejemplo

**Entrada:**

- ADN: `ACGTACGT`
- Motivos: `ACG`, `CGT`, `TAC`
- $k = 2$

**Ocurrencias:**

- `ACG`: [0,2], [4,6]
- `CGT`: [1,3], [5,7]
- `TAC`: [3,5]

**Ventana candidata:**

$$[0,3] = \texttt{ACGT}$$

- Contiene `ACG` (0–2)
- Contiene `CGT` (1–3)

# L. Descubriendo Genómicos — Ejemplo

**Entrada:**

- ADN: `ACGTACGT`
- Motivos: `ACG`, `CGT`, `TAC`
- $k = 2$

**Ocurrencias:**

- `ACG`: [0,2], [4,6]
- `CGT`: [1,3], [5,7]
- `TAC`: [3,5]

**Ventana candidata:**

$$[0, 3] = \texttt{ACGT}$$

- Contiene `ACG` (0–2)
- Contiene `CGT` (1–3)

$$\Rightarrow \text{longitud} = 4$$

# L. Descubriendo Genómicos — Ejemplo

**Entrada:**

- ADN: `ACGTACGT`
- Motivos: `ACG`, `CGT`, `TAC`
- $k = 2$

**Ocurrencias:**

- `ACG`: [0,2], [4,6]
- `CGT`: [1,3], [5,7]
- `TAC`: [3,5]

**Ventana candidata:**

$$[0,3] = \texttt{ACGT}$$

- Contiene `ACG` (0–2)
- Contiene `CGT` (1–3)

$$\Rightarrow \text{longitud} = 4$$

**Resultado:** existen varias ventanas óptimas; la longitud mínima es 4.

# L. Descubriendo Genómicos — Pseudocódigo

## Idea general

Trabajar con ocurrencias en lugar de generar todas las subcadenas.

# L. Descubriendo Genómicos — Pseudocódigo

## Idea general

Trabajar con ocurrencias en lugar de generar todas las subcadenas.

```
1. Eliminar motivos duplicados

2. Para cada motivo:
       buscar todas sus ocurrencias en el ADN
       guardar (fin, inicio, idMotivo)

3. Ordenar ocurrencias por fin (R)

4. Inicializar latestStart[i] = -1

5. Para cada ocurrencia (R, L, id):
       actualizar latestStart[id] = max(...)

       recoger inicios activos

       si hay al menos k:
           ordenar inicios desc
```

# L. Descubriendo Genómicos — Pseudocódigo

## Idea general

Trabajar con ocurrencias en lugar de generar todas las subcadenas.

```
1. Eliminar motivos duplicados

2. Para cada motivo:
       buscar todas sus ocurrencias en el ADN
       guardar (fin, inicio, idMotivo)

3. Ordenar ocurrencias por fin (R)

4. Inicializar latestStart[i] = -1

5. Para cada ocurrencia (R, L, id):
       actualizar latestStart[id] = max(...)

       recoger inicios activos

       si hay al menos k:
           ordenar inicios desc
```

# M. La cascada de copas

| Envíos | Válidos | % éxito |
|---|---|---|
| 2 | 1 | 50 % |

# M. La cascada de copas — Resumen

## ¿Qué se pide?

Dada una pirámide de copas (dadas por nivel de altura, fila y columna), su capacidad y un volumen de líquido sobre la primera copa ("**entrada**") y un listado de copas (dadas por nivel, fila y columna) ("**consulta**"), devolver: **volumen de líquido** de cada copa de la **consulta**.

# M. La cascada de copas — Resumen

## ¿Qué se pide?

Dada una pirámide de copas (dadas por nivel de altura, fila y columna), su capacidad y un volumen de líquido sobre la primera copa ("**entrada**") y un listado de copas (dadas por nivel, fila y columna) ("**consulta**"), devolver: **volumen de líquido** de cada copa de la **consulta**.

## Observación clave

El volumen de líquido que llega a una copa solo depende de las 4 copas que se apoyan en ella (recursivamente).

# M. La cascada de copas — Resumen

## ¿Qué se pide?

Dada una pirámide de copas (dadas por nivel de altura, fila y columna), su capacidad y un volumen de líquido sobre la primera copa ("**entrada**") y un listado de copas (dadas por nivel, fila y columna) ("**consulta**"), devolver: **volumen de líquido** de cada copa de la **consulta**.

## Observación clave

El volumen de líquido que llega a una copa solo depende de las 4 copas que se apoyan en ella (recursivamente).

## Problema de la solución exhaustiva

Si la piramide es grande y la lista de consultas es muy grande, se hacen muchos cálculos redundantes.

# M. La cascada de copas — Resumen

## ¿Qué se pide?

Dada una pirámide de copas (dadas por nivel de altura, fila y columna), su capacidad y un volumen de líquido sobre la primera copa ("**entrada**") y un listado de copas (dadas por nivel, fila y columna) ("**consulta**"), devolver: **volumen de líquido** de cada copa de la **consulta**.

## Observación clave

El volumen de líquido que llega a una copa solo depende de las 4 copas que se apoyan en ella (recursivamente).

## Problema de la solución exhaustiva

Si la piramide es grande y la lista de consultas es muy grande, se hacen muchos cálculos redundantes.

## Idea principal:

En lugar de hacer la recursión desde abajo, es mejor construir el flujo de cada copa una sola vez en una memoria (programación dinámica)

# M. La cascada de copas — Solución

**Paso 1: Estructura de Datos**
Almacenamos las capacidades en un array bidimensional $C[h][i]$. El número de copas en el nivel $h$ es $(h+1)^2$.

$$\text{Índice lineal: } i = \mathit{fila} \times (h+1) + \mathit{columna} \quad \text{donde } \mathit{fila}, \mathit{columna} \in [0, h]$$

**Paso 2: Simulación de Flujo**
Calculamos iterativamente el flujo acumulado $F[h][i]$. Para cada copa con $F[h][i] > C[h][i]$, el sobrante $S = F[h][i] - C[h][i]$ se reparte equitativamente entre sus cuatro sucesoras en el nivel $h+1$:

- `Arriba-Izq`: $\mathit{fila}, \mathit{columna}$
- `Arriba-Der`: $\mathit{fila}+1, \mathit{columna}$
- `Abajo-Izq`: $\mathit{fila}, \mathit{columna}+1$
- `Abajo-Der`: $\mathit{fila}+1, \mathit{columna}+1$

**Paso 3: Resolución de Consultas**
El líquido contenido en una copa es: $L[h][i] = \text{mín}(C[h][i], F[h][i])$.

# M. La cascada de copas — Solución

**Paso 1: Estructura de Datos**
Almacenamos las capacidades en un array bidimensional $C[h][i]$. El número de copas en el nivel $h$ es $(h+1)^2$.

$$\text{Índice lineal: } i = \mathit{fila} \times (h+1) + \mathit{columna} \quad \text{donde } \mathit{fila}, \mathit{columna} \in [0, h]$$

**Paso 2: Simulación de Flujo**
Calculamos iterativamente el flujo acumulado $F[h][i]$. Para cada copa con $F[h][i] > C[h][i]$, el sobrante $S = F[h][i] - C[h][i]$ se reparte equitativamente entre sus cuatro sucesoras en el nivel $h+1$:

- `Arriba-Izq`: $\mathit{fila}, \mathit{columna}$
- `Arriba-Der`: $\mathit{fila}+1, \mathit{columna}$
- `Abajo-Izq`: $\mathit{fila}, \mathit{columna}+1$
- `Abajo-Der`: $\mathit{fila}+1, \mathit{columna}+1$

**Paso 3: Resolución de Consultas**
El líquido contenido en una copa es: $L[h][i] = \text{mín}(C[h][i], F[h][i])$.

# M. La cascada de copas — Modelado Visual

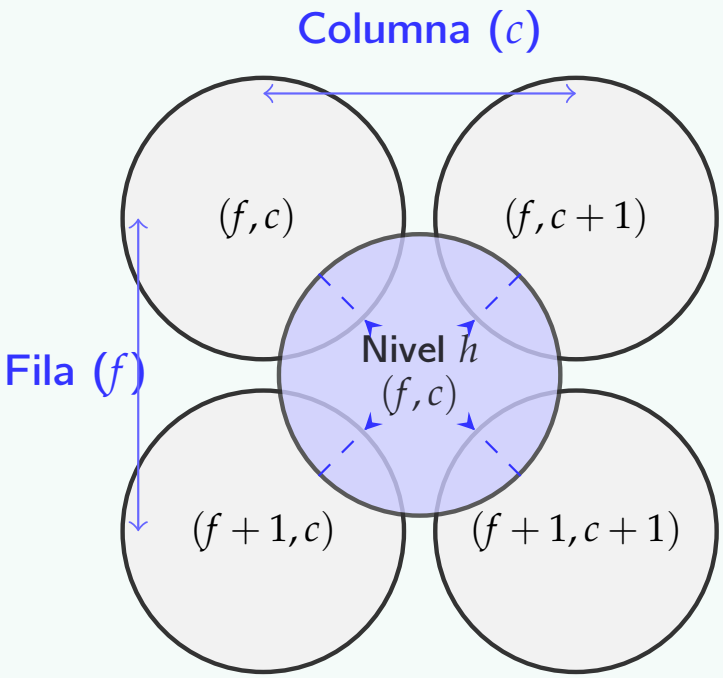



Figura: Visualización de la proyección de flujo de una copa $(f,c)$ en el nivel $h$ a sus cuatro sucesoras en el nivel $h+1$.

## G. La muralla del reino

| Envíos | Válidos | % éxito |
|---|---|---|
| 6 | 3 | 50 % |

# G. La muralla del reino — Resumen

## ¿Qué se pide?

Reconstruir la muralla en el menor tiempo posible, repartiendo los tiempos de reconstrucción de los $N$ tramos $(h_1, h_2, \ldots, h_N)$ entre no más de $K$ brigadas de obreros, sin alterar el orden original de los tramos y asumiendo que todas las brigadas empiezan su trabajo a la vez.

# G. La muralla del reino — Resumen

## ¿Qué se pide?

Reconstruir la muralla en el menor tiempo posible, repartiendo los tiempos de reconstrucción de los $N$ tramos $(h_1, h_2, \ldots, h_N)$ entre no más de $K$ brigadas de obreros, sin alterar el orden original de los tramos y asumiendo que todas las brigadas empiezan su trabajo a la vez.

## Con otras palabras

Encontrar una *partición lineal* de la secuencia $h_1, h_2, \ldots, h_N$ en $K$ subsecuencias, tal que la mayor de las $K$ sumas sea la mínima posible.

# G. La muralla del reino — Resumen

## ¿Qué se pide?

Reconstruir la muralla en el menor tiempo posible, repartiendo los tiempos de reconstrucción de los $N$ tramos $(h_1, h_2, \ldots, h_N)$ entre no más de $K$ brigadas de obreros, sin alterar el orden original de los tramos y asumiendo que todas las brigadas empiezan su trabajo a la vez.

## Con otras palabras

Encontrar una *partición lineal* de la secuencia $h_1, h_2, \ldots, h_N$ en $K$ subsecuencias, tal que la mayor de las $K$ sumas sea la mínima posible.

## Restricciones

- $1 \leq K \leq N \leq 10^5$
- $0 \leq h_i \leq 10^9$

# G. La muralla del reino— Backtracking

## Resolución

- Generar y evaluar una por una todas las particiones posibles para encontrar la óptima.
- Una partición válida se genera eligiendo $K-1$ puntos de corte entre tramos de las $N-1$ posiciones posibles: el primer punto de corte después del tramo $t$ y recursivamente los demás $K-2$ puntos de corte entre los $N-t$ tramos restantes.
- Una partición se evalúa sumando los tiempos de los tramos asignados a cada brigada por separado y seleccionando la carga (suma) máxima.
- Por último, de estas cargas máximas de todas las particiones posibles, se selecciona la menor y tendremos el tiempo mínimo en que estará reconstruida la muralla.

# G. La muralla del reino— Backtracking

## Recursividad

- $r(t,K) \rightarrow$ Máxima carga de la partición óptima de los tramos $[t,N]$ con $K$ brigadas.
- Si $t = N \rightarrow$ Asignar el último tramo a una de las $K$ brigadas disponibles.

$$r(N,K) = h_N$$

- Si $K = 1 \rightarrow$ Asignar todos los tramos $[t,N]$ a la única brigada disponible.

$$r(t,1) = \sum_{i=t}^{N} h_i$$

- En otro caso $\rightarrow$ Asignar a la 1ª brigada los tramos $[t,i]$, donde $t \leq i < N$, y los tramos $[i+1,N]$ a las $K-1$ brigadas restantes.

$$r(t,K) = \min_{i=t}^{N-1} \left\{ \text{máx} \left( \sum_{j=t}^{i} h_j \ , \ r(i+1,K-1) \right) \right\}$$

# G. La muralla del reino— Backtracking

## Complejidad

- El algoritmo selecciona $K-1$ puntos de corte de las $N-1$ posibilidades que existen. Por tanto, para encontrar la partición óptima evalúa

$$\binom{N-1}{K-1} = \frac{(N-1)!}{(K-1)!(N-K)!} \text{ combinaciones}$$

- Puesto que hay que sumar cada grupo de tramos para hallar la suma mayor, la complejidad es:

$$\mathcal{O}\left(N \cdot \binom{N-1}{K-1}\right)$$

# G. La muralla del reino— Backtracking

## Complejidad

- Pueden resolverse casos pequeños, por ejemplo, $N = 15$ y $K = 6$:

$$15 \cdot \binom{14}{5} = 30\,030 \text{ operaciones} \rightarrow \text{Entra en el tiempo del juez}$$

- Para casos de mayor tamaño el tiempo se dispara muy rápidamente, por ejemplo:

$$61 \cdot \binom{60}{15} = 3,24 \cdot 10^{15} \text{ operaciones} \rightarrow \text{¡TLE!}$$

# G. La muralla del reino— Programación dinámica

## Tabla de subproblemas

- $sp(k,t) \rightarrow$ Máxima carga de la partición óptima de los primeros $t$ tramos con $k$ brigadas:

$$sp(t,k) = \min_{i=1}^{t}\{\max(sp(k-1,i), \sum_{j=i+1}^{t} h_j)\}$$

- El mejor resultado para $t$ tramos y $k$ brigadas se encuentra probando todas las posiciones $i$ posibles para el último corte, comparando la carga de la última brigada con la solución óptima de las $k-1$ brigadas anteriores.

# G. La muralla del reino— Programación dinámica

## Tabla de subproblemas

- $sp(k,t) \rightarrow$ Máxima carga de la partición óptima de los primeros $t$ tramos con $k$ brigadas:

$$sp(t,k) = \min_{i=1}^{t}\{\max(sp(k-1,i), \sum_{j=i+1}^{t} h_j)\}$$

- El mejor resultado para $t$ tramos y $k$ brigadas se encuentra probando todas las posiciones $i$ posibles para el último corte, comparando la carga de la última brigada con la solución óptima de las $k-1$ brigadas anteriores.

## Resolución

Se rellena la tabla en el orden de filas y columnas y la solución óptima se encuentra en la última posición, $sp(K,N)$.

# G. La muralla del reino— Programación dinámica

## Complejidad

- La suma de los últimos tramos, $\sum_{j=i+1}^{t} h_j$, se puede calcular mediante prefijos para que se realice en tiempo constante $\mathcal{O}(1)$.

# G. La muralla del reino— Programación dinámica

## Complejidad

- La suma de los últimos tramos, $\sum_{j=i+1}^{t} h_j$, se puede calcular mediante prefijos para que se realice en tiempo constante $\mathcal{O}(1)$.
- El recorrido de la tabla de subproblemas requiere un tiempo $\mathcal{O}(K \cdot N)$.

# G. La muralla del reino— Programación dinámica

## Complejidad

- La suma de los últimos tramos, $\sum_{j=i+1}^{t} h_j$, se puede calcular mediante prefijos para que se realice en tiempo constante $\mathcal{O}(1)$.
- El recorrido de la tabla de subproblemas requiere un tiempo $\mathcal{O}(K \cdot N)$.
- La elección del último punto de corte, $i$, es $\mathcal{O}(N)$.

# G. La muralla del reino— Programación dinámica

## Complejidad

- La suma de los últimos tramos, $\sum_{j=i+1}^{t} h_j$, se puede calcular mediante prefijos para que se realice en tiempo constante $\mathcal{O}(1)$.
- El recorrido de la tabla de subproblemas requiere un tiempo $\mathcal{O}(K \cdot N)$.
- La elección del último punto de corte, $i$, es $\mathcal{O}(N)$.
- Por tanto, la complejidad final es $\mathcal{O}(K \cdot N^2)$.

# G. La muralla del reino— Programación dinámica

## Complejidad

- La suma de los últimos tramos, $\sum_{j=i+1}^{t} h_j$, se puede calcular mediante prefijos para que se realice en tiempo constante $\mathcal{O}(1)$.
- El recorrido de la tabla de subproblemas requiere un tiempo $\mathcal{O}(K \cdot N)$.
- La elección del último punto de corte, $i$, es $\mathcal{O}(N)$.
- Por tanto, la complejidad final es $\mathcal{O}(K \cdot N^2)$.

Supone una mejora importante respecto al *backtracking*. Aún así, no supera los casos de prueba grandes del juez. ¡TLE!

# G. La muralla del reino— Búsqueda binaria

## Adivinar la respuesta

- En lugar de buscar dónde establecer los puntos de corte entre tramos, «jugamos a adivinar» la respuesta.
- ¿Es posible repartir los $N$ tramos de la muralla entre $K$ brigadas de modo que ninguna tarde más de $X$ horas en hacer su trabajo?

# G. La muralla del reino— Búsqueda binaria

## Adivinar la respuesta

- En lugar de buscar dónde establecer los puntos de corte entre tramos, «jugamos a adivinar» la respuesta.
- ¿Es posible repartir los $N$ tramos de la muralla entre $K$ brigadas de modo que ninguna tarde más de $X$ horas en hacer su trabajo?

## Respuesta monótona

Si se puede reconstruir la muralla en un tiempo $X$ con $K$ brigadas, entonces también se puede hacer con más brigadas.

# G. La muralla del reino— Búsqueda binaria

## Adivinar la respuesta

- En lugar de buscar dónde establecer los puntos de corte entre tramos, «jugamos a adivinar» la respuesta.
- ¿Es posible repartir los $N$ tramos de la muralla entre $K$ brigadas de modo que ninguna tarde más de $X$ horas en hacer su trabajo?

## Respuesta monótona

Si se puede reconstruir la muralla en un tiempo $X$ con $K$ brigadas, entonces también se puede hacer con más brigadas.

## Resolución del problema

Por lo tanto, podemos **explorar el espacio de soluciones** del problema mediante **búsqueda binaria** para hallar la respuesta correcta.

# G. La muralla del reino— Búsqueda binaria

## Rango de búsqueda

- **Límite inferior**: $\text{máx}(h_1, h_2, \ldots, h_N)$. Es imposible terminar antes la obra, ya que los tramos de muralla no se pueden trocear.
- **Límite superior**: $\sum_{i=1}^{N} h_i$. Escenario en que una brigada reconstruye toda la muralla ($K = 1$).

# G. La muralla del reino— Búsqueda binaria

## Rango de búsqueda

- **Límite inferior**: $\text{máx}(h_1, h_2, \ldots, h_N)$. Es imposible terminar antes la obra, ya que los tramos de muralla no se pueden trocear.
- **Límite superior**: $\sum_{i=1}^{N} h_i$. Escenario en que una brigada reconstruye toda la muralla ($K = 1$).

## Validación de candidatos

- ¿Se puede reparar la muralla en $X$ horas?
- **Algoritmo greedy**: Sumar, en el orden dado, los tiempos de los tramos que se asignarán a la primera brigada. Si al sumar un valor $h_i$ se supera $X$, se asigna este valor a la siguiente brigada. Se continúa con la asignación de tramos y si al final hay suficientes brigadas para asignar todos los tramos, entonces $X$ es un candidato válido.

# G. La muralla del reino— Búsqueda binaria

## Complejidad

- **Búsqueda binaria**: Se realiza sobre el rango $[\text{máx}(h_i), \sum h_i]$, por lo que

$$\mathcal{O}\left(\log_2 \sum_{i=1}^{N} h_i\right)$$

# G. La muralla del reino— Búsqueda binaria

## Complejidad

- **Búsqueda binaria**: Se realiza sobre el rango $[\text{máx}(h_i), \sum h_i]$, por lo que

$$\mathcal{O}\left(\log_2 \sum_{i=1}^{N} h_i\right)$$

- **Validación**: Recorrido completo de los $h_i$ en el peor caso, entonces $\mathcal{O}(N)$ por cada candidato.

# G. La muralla del reino— Búsqueda binaria

## Complejidad

- **Búsqueda binaria**: Se realiza sobre el rango $[\text{máx}(h_i), \sum h_i]$, por lo que

$$\mathcal{O}\left(\log_2 \sum_{i=1}^{N} h_i\right)$$

- **Validación**: Recorrido completo de los $h_i$ en el peor caso, entonces $\mathcal{O}(N)$ por cada candidato.
- **En total**:

$$\mathcal{O}\left(N \cdot \log_2 \sum_{i=1}^{N} h_i\right)$$

- Incluso en los casos de prueba extremos del juez, la estrategia de búsqueda binaria elude perfectamente el veredicto TLE.

# G. La muralla del reino— Fallos comunes

- **Desbordamiento**: No usar `long long` en C++ o `long` en Java para las sumas de los $h_i$ y para las variables de control de la búsqueda binaria.

# G. La muralla del reino— Fallos comunes

- **Desbordamiento**: No usar `long long` en C++ o `long` en Java para las sumas de los $h_i$ y para las variables de control de la búsqueda binaria.
- **Límite inferior**: Iniciarlo en un valor incorrecto, como 0 o el promedio de los $h_i$, hará que la validación falle cuando intente asignar un tramo con $h_i$ superior al valor candidato.

# G. La muralla del reino— Fallos comunes

- **Desbordamiento**: No usar `long long` en C++ o `long` en Java para las sumas de los $h_i$ y para las variables de control de la búsqueda binaria.
- **Límite inferior**: Iniciarlo en un valor incorrecto, como 0 o el promedio de los $h_i$, hará que la validación falle cuando intente asignar un tramo con $h_i$ superior al valor candidato.
- **Límites de la búsqueda**: Actualizarlos mal puede provocar un bucle infinito o un resultado erróneo.

# I. Numble

| Envíos | Válidos | % éxito |
|---|---|---|
| 0 | 0 | – |

# I. Numble — Resumen

## ¿Qué se pide?

Hay una secuencia secreta de longitud $L$, con valores entre $1$ y $N$.
En cada intento proponemos otra secuencia y el juez devuelve:

- **verdes**: números correctos en posición correcta;
- **amarillas**: números correctos en posición incorrecta.

Debemos adivinar la secuencia en como mucho $Q$ intentos.

# I. Numble — Resumen

## ¿Qué se pide?

Hay una secuencia secreta de longitud $L$, con valores entre $1$ y $N$.
En cada intento proponemos otra secuencia y el juez devuelve:

- **verdes**: números correctos en posición correcta;
- **amarillas**: números correctos en posición incorrecta.

Debemos adivinar la secuencia en como mucho $Q$ intentos.

## Restricciones

- $1 \leq L \leq 6$
- $1 \leq N \leq 6$
- $1 \leq Q \leq 20$

# I. Numble — Resumen

## ¿Qué se pide?

Hay una secuencia secreta de longitud $L$, con valores entre $1$ y $N$.
En cada intento proponemos otra secuencia y el juez devuelve:

- **verdes**: números correctos en posición correcta;
- **amarillas**: números correctos en posición incorrecta.

Debemos adivinar la secuencia en como mucho $Q$ intentos.

## Restricciones

- $1 \leq L \leq 6$
- $1 \leq N \leq 6$
- $1 \leq Q \leq 20$

## Observación clave

El número total de secuencias posibles es:

$$N^L \leq 6^6 = 46656$$

Así que podemos intentar generar todas las secuencias posibles.

# I. Numble — Espacio de candidatos

Mantenemos un conjunto $C$ de secuencias **consistentes** con todas las respuestas recibidas hasta el momento.

# I. Numble — Espacio de candidatos

Mantenemos un conjunto $C$ de secuencias **consistentes** con todas las respuestas recibidas hasta el momento.
Inicialmente:

$$C = \{\text{todas las secuencias posibles}\}$$

# I. Numble — Espacio de candidatos

Mantenemos un conjunto $C$ de secuencias **consistentes** con todas las respuestas recibidas hasta el momento.
Inicialmente:

$$C = \{\text{todas las secuencias posibles}\}$$

Tras hacer una consulta `guess` y recibir un feedback $(v, a)$, filtramos:

$$C' = \{x \in C : \mathit{feedback}(x, \texttt{guess}) = (v, a)\}$$

# I. Numble — Espacio de candidatos

Mantenemos un conjunto $C$ de secuencias **consistentes** con todas las respuestas recibidas hasta el momento.
Inicialmente:

$$C = \{\text{todas las secuencias posibles}\}$$

Tras hacer una consulta `guess` y recibir un feedback $(v, a)$, filtramos:

$$C' = \{x \in C : \mathit{feedback}(x, \texttt{guess}) = (v, a)\}$$

## Idea general

Una forma clásica de resolver esto es usar **minmax** sobre $C$: elegir la consulta que minimiza el tamaño del grupo más grande tras particionar por posibles respuestas.

# I. Numble — Espacio de candidatos

Mantenemos un conjunto $C$ de secuencias **consistentes** con todas las respuestas recibidas hasta el momento.
Inicialmente:

$$C = \{\text{todas las secuencias posibles}\}$$

Tras hacer una consulta `guess` y recibir un feedback $(v, a)$, filtramos:

$$C' = \{x \in C : \mathit{feedback}(x, \texttt{guess}) = (v, a)\}$$

## Idea general

Una forma clásica de resolver esto es usar **minmax** sobre $C$: elegir la consulta que minimiza el tamaño del grupo más grande tras particionar por posibles respuestas.

Eso funciona, pero puede ser innecesariamente costoso si lo aplicamos desde el principio sobre todo el conjunto.

# I. Numble — Primer paso: descubrir los números

Como $N \leq 6$, primero podemos probar las secuencias:

$$111111,\ 222222,\ 333333,\ 444444,\ 555555,\ 666666$$

truncadas a longitud $L$ si hace falta.

# I. Numble — Primer paso: descubrir los números

Como $N \leq 6$, primero podemos probar las secuencias:

$$111111,\ 222222,\ 333333,\ 444444,\ 555555,\ 666666$$

truncadas a longitud $L$ si hace falta.

**¿Qué obtenemos?**

En una consulta de la forma `dddddd`, todos los aciertos de ese dígito aparecen como:

$$verdes + amarillas$$

porque toda la secuencia está formada por el mismo número.

# I. Numble — Primer paso: descubrir los números

Como $N \leq 6$, primero podemos probar las secuencias:

$$111111,\ 222222,\ 333333,\ 444444,\ 555555,\ 666666$$

truncadas a longitud $L$ si hace falta.

**¿Qué obtenemos?**

En una consulta de la forma `dddddd`, todos los aciertos de ese dígito aparecen como:

$$verdes + amarillas$$

porque toda la secuencia está formada por el mismo número.

Por tanto, tras como mucho $N \leq 6$ consultas sabemos exactamente cuántas veces aparece cada número en la solución.

# I. Numble — Reducción del espacio

Supongamos que, tras esas consultas, sabemos que el número $i$ aparece $c_i$ veces en la secuencia secreta.

Entonces, el número de secuencias compatibles ya no es $N^L$, sino:

$$\frac{L!}{c_1!c_2!\cdots c_N!}$$

# I. Numble — Reducción del espacio

Supongamos que, tras esas consultas, sabemos que el número $i$ aparece $c_i$ veces en la secuencia secreta.

Entonces, el número de secuencias compatibles ya no es $N^L$, sino:

$$\frac{L!}{c_1!c_2!\cdots c_N!}$$

## Peor caso

Esta cantidad se maximiza cuando todos los números son distintos, es decir, cuando todos los $c_i$ son 0 o 1. En ese caso:

$$\frac{L!}{c_1!c_2!\cdots c_N!} = L! \leq 6! = 720$$

# I. Numble — Reducción del espacio

Supongamos que, tras esas consultas, sabemos que el número $i$ aparece $c_i$ veces en la secuencia secreta.
Entonces, el número de secuencias compatibles ya no es $N^L$, sino:

$$\frac{L!}{c_1!c_2!\cdots c_N!}$$

## Peor caso

Esta cantidad se maximiza cuando todos los números son distintos, es decir, cuando todos los $c_i$ son 0 o 1. En ese caso:

$$\frac{L!}{c_1!c_2!\cdots c_N!} = L! \leq 6! = 720$$

## Conclusión

Gastando como mucho 6 intentos, reducimos el espacio de búsqueda de hasta

$$46656$$

candidatos a, como mucho,

$$720$$

# I. Numble — Cómo terminar

## Opción 1: minmax

Con el conjunto reducido, ya podemos aplicar una estrategia tipo Knuth/minmax: para cada posible consulta, miramos cómo partiría el conjunto $C$ según la respuesta obtenida y elegimos la que deja el peor grupo lo más pequeño posible.

# I. Numble — Cómo terminar

## Opción 1: minmax

Con el conjunto reducido, ya podemos aplicar una estrategia tipo Knuth/minmax: para cada posible consulta, miramos cómo partiría el conjunto $C$ según la respuesta obtenida y elegimos la que deja el peor grupo lo más pequeño posible.

## Opción 2: aleatoria

También podemos elegir el siguiente candidato **aleatoriamente** dentro de $C$. Como el conjunto restante ya es pequeño, esta estrategia también funciona bien en la práctica y entra dentro del límite de intentos.

## F. El historial de AdaByron

| Envíos | Válidos | % éxito |
|---|---|---|
| 9 | 1 | 11 % |

# F. El historial de AdaByron

## Resumen

Mantener varias versiones de un array de tamaños de problemas, permitiendo:

- **Update** (`U`): crear una nueva versión cambiando $a_i = x$.
- **Sum** (`S`): consultar la suma en un rango $[l, r]$ de una versión concreta.
- **Objetivo:** responder $Q$ $(U + S)$ operaciones $(Q \leq 3 \cdot 10^5)$ eficientemente.

La versión 0 es la inicial y cada operación `U` crea una nueva versión a partir de la más reciente.

# F. El historial de AdaByron

## Versión básica

### Idea básica

- Guardar cada versión como un **array completo**.

**Consulta** `S v l r`: recorrer $[l, r]$ y sumar.

- Tiempo total ($S$ consultas): $O(S \cdot N)$ KO

**Actualizaciones** `U i x`: copiar la última versión y modificar $a[i] = x$.

$$\begin{array}{c} \text{Versión 0: } [100, 200, 150, 300, 250] \\ \downarrow \; U(2, 500) \\ \text{Versión 1: } [100, 500, 150, 300, 250] \end{array}$$

- Tiempo total ($U$ actualizaciones): $O(U \cdot N)$ KO
- Memoria total ($U$ actualizaciones): $O(U \cdot N)$ KO

# F. El historial de AdaByron

**Versión mejorada**

## Idea básica

- Guardar cada versión como un **array de sumas prefijo**.
- Definimos:

$$sum[i] = a[1] + a[2] + \cdots + a[i]$$

**Consulta** `S v l r`: usar restas de prefijos.

$$a[l] + \cdots + a[r] = sum[r] - sum[l-1]$$

- Tiempo total ($S$ consultas): $O(S \cdot 1)$ OK

**Ejemplo:**

$$\text{Array: } [100, 200, 150, 300, 250]$$
$$\text{Prefijos: } [100, 300, 450, 750, 1000]$$

# F. El historial de AdaByron

**Versión mejorada**
**Actualización** `U i x`: copiar la versión y rehacer las sumas desde $i$ hasta $N$.

$$\begin{array}{c} \text{Versión 0: } [100, 300, 450, 750, 1000] \\ \downarrow \; U(2, 500) \\ \text{Versión 1: } [100, 600, 750, 1050, 1300] \end{array}$$

- Tiempo total ($U$ actualizaciones): $O(U \cdot N)$ KO
- Memoria total ($U$ actualizaciones): $O(U \cdot N)$ KO

## Conclusión

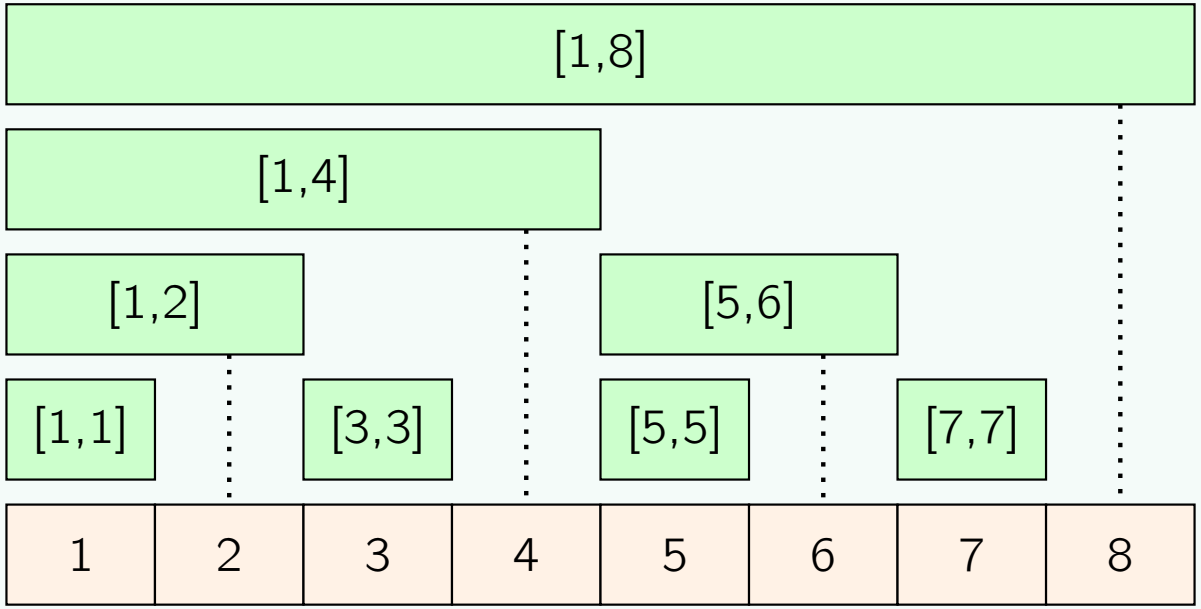
Mejoramos mucho las consultas, pero las actualizaciones siguen siendo demasiado caras.

# F. El historial de AdaByron

**Versión válida**

## Idea básica

- Usar un **Fenwick Tree** para mantener sumas prefijo.
- Cada posición $bit[i]$ guarda la suma del intervalo: $[i - lowbit(i) + 1,\ i]$




- Consulta prefijo: recorrer $O(\log N)$ posiciones.
- Update: modificar $O(\log N)$ posiciones.

# F. El historial de AdaByron

**Versión válida**

## Persistencia

En lugar de guardar un único valor en $bit[i]$, guardamos una lista de pares `(version, valor)`.

**Consulta** `S v l r`:

- Se recorren $O(\log N)$ posiciones del Fenwick.
- En cada una, se busca el valor válido para la versión $v$ (búsqueda binaria).
- Tiempo total ($S$ consultas): $O(S \cdot \log N \cdot \log Q)$ OK

**Actualización** `U i x`:

- Calcular el incremento $\Delta = x - a_i$.
- Actualizar las $O(\log N)$ posiciones del Fenwick.
- En cada una, añadir un nuevo par `(version, valor)`.
- Tiempo total ($U$ actualizaciones): $O(U \cdot \log N)$ OK
- Memoria total ($U$ actualizaciones): $O(U \cdot \log N)$ OK

## Conclusión

Primera solución **válida**: evita copiar arrays completos y mantiene todas las versiones.

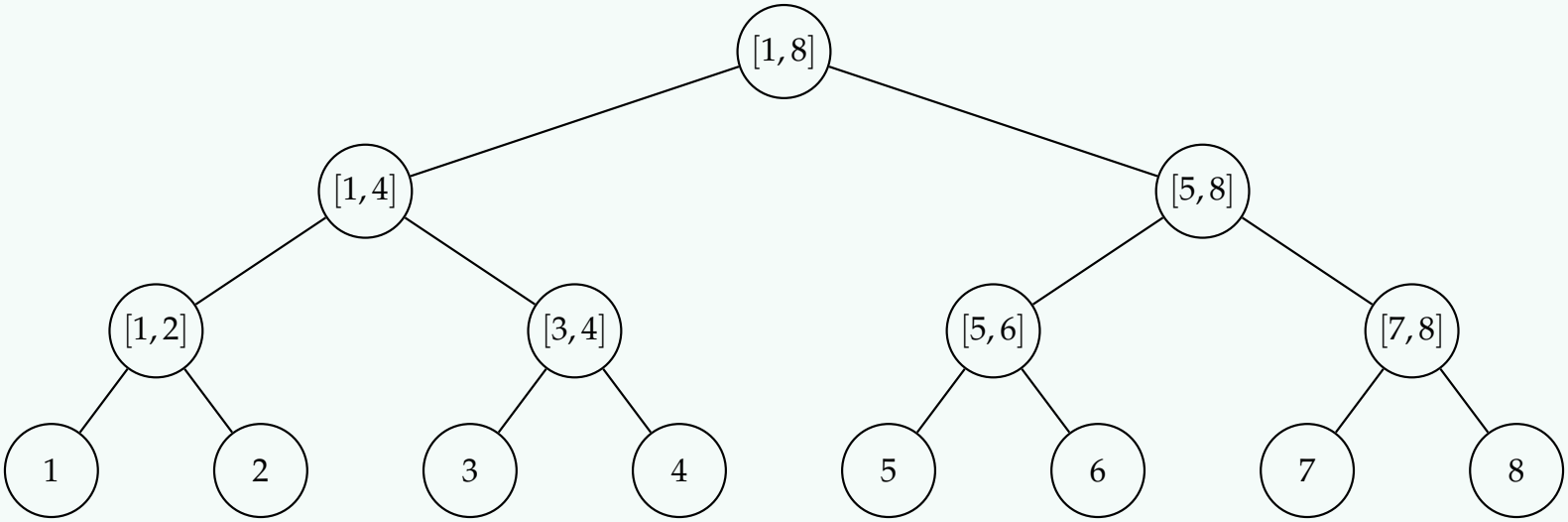
# F. El historial de AdaByron

## Versión óptima

### Idea básica

- Un **Segment Tree** divide el array en intervalos.
- Cada nodo guarda la suma de un rango.
- Permite:
  - consultar sumas en rango
  - actualizar una posición

en $O(\log N)$.

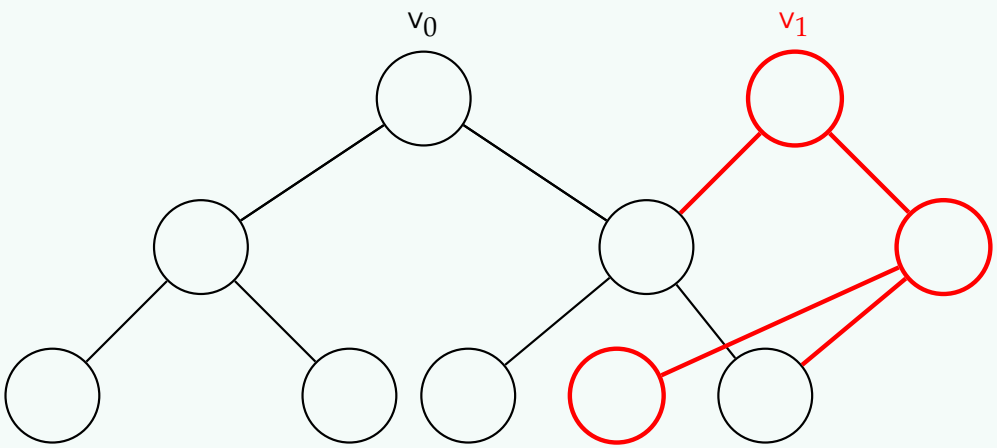


# F. El historial de AdaByron

**Versión óptima**

## Persistencia

- Cada actualización crea una nueva raíz.
- Solo se copian los nodos del camino raíz-hoja.
- El resto del árbol se comparte.




- Negro: nodos compartidos entre versiones.
- Rojo: nodos nuevos tras la actualización.

# F. El historial de AdaByron

**Versión óptima**

**Consulta** `S v l r`:

- Usar la raíz de la versión $v$.
- Hacer una suma en rango sobre el segment tree.
- Tiempo total ($S$ consultas): $O(S \cdot \log N)$ OK

**Actualización** `U i x`:

- Crear una nueva raíz.
- Copiar solo el camino raíz-hoja de la posición $i$.
- Tiempo total ($U$ actualizaciones): $O(U \cdot \log N)$ OK
- Memoria total ($U$ actualizaciones): $O(U \cdot \log N)$ OK

## Conclusión

El **segment tree persistente** da la solución óptima para este problema.

## C. El Circo de la Luna

| Envíos | Válidos | % éxito |
|---|---|---|
| 0 | 0 | – |

# C. El Circo de la Luna — Resumen

## ¿Qué se pide?

En cada fin de semana hay que elegir exactamente uno de dos espectáculos: **A** o **B**.
Queremos maximizar el número de fines de semana consecutivos que pueden programarse desde el primero, cumpliendo:

- cada tipo de polvo se puede usar como mucho una vez;
- si en una semana usamos $u$ polvos inestables, la semana siguiente debe tener al menos $u$ polvos neutralizantes.

# C. El Circo de la Luna — Resumen

## ¿Qué se pide?

En cada fin de semana hay que elegir exactamente uno de dos espectáculos: **A** o **B**.
Queremos maximizar el número de fines de semana consecutivos que pueden programarse desde el primero, cumpliendo:

- cada tipo de polvo se puede usar como mucho una vez;
- si en una semana usamos $u$ polvos inestables, la semana siguiente debe tener al menos $u$ polvos neutralizantes.

## Restricciones

- $1 \leq T \leq 250$
- $1 \leq M \leq 10^4$
- $1 \leq P \leq 2 \cdot 10^4$
- $0 \leq I, N \leq P$
- cada espectáculo usa entre 1 y 10 polvos

# C. El Circo de la Luna — Simplificación

## Paso 1: convertirlo en un problema de decisión

En vez de buscar directamente la respuesta final, fijemos un valor $k$ y preguntémonos:

¿Se pueden programar las primeras $k$ semanas?

Si sabemos resolver esta pregunta, luego ya podremos encontrar el mayor $k$.

# C. El Circo de la Luna — Simplificación

## Paso 1: convertirlo en un problema de decisión

En vez de buscar directamente la respuesta final, fijemos un valor $k$ y preguntémonos:

¿Se pueden programar las primeras $k$ semanas?

Si sabemos resolver esta pregunta, luego ya podremos encontrar el mayor $k$.

## Modelado con 2-SAT

Para cada semana $i$, definimos una variable booleana $x_i$:

- $x_i =$ `True`: elegimos el espectáculo **A**
- $x_i =$ `False`: elegimos el espectáculo **B**

Así, una asignación de valores a las variables representa una elección de espectáculos para las primeras $k$ semanas.

# C. El Circo de la Luna — Restricciones como 2-SAT

Para cada semana $i$, tenemos dos posibles elecciones:

$$A_i \qquad \text{ó} \qquad B_i$$

Lo representamos con una variable booleana:

$$x_i = \texttt{True} \iff A_i, \qquad x_i = \texttt{False} \iff B_i$$

# C. El Circo de la Luna — Restricciones como 2-SAT

Para cada semana $i$, tenemos dos posibles elecciones:

$$A_i \qquad \text{ó} \qquad B_i$$

Lo representamos con una variable booleana:

$$x_i = \texttt{True} \iff A_i, \qquad x_i = \texttt{False} \iff B_i$$

## Restricciones de incompatibilidad

Si dos elecciones $X$ e $Y$ no pueden ocurrir a la vez, añadimos la cláusula:

$$\neg(X \wedge Y)$$

que en 2-SAT es equivalente a:

$$X \Rightarrow \neg Y, \qquad Y \Rightarrow \neg X$$

# C. El Circo de la Luna — Restricciones como 2-SAT

Para cada semana $i$, tenemos dos posibles elecciones:

$$A_i \qquad \text{ó} \qquad B_i$$

Lo representamos con una variable booleana:

$$x_i = \texttt{True} \iff A_i, \qquad x_i = \texttt{False} \iff B_i$$

## Restricciones de incompatibilidad

Si dos elecciones $X$ e $Y$ no pueden ocurrir a la vez, añadimos la cláusula:

$$\neg(X \wedge Y)$$

que en 2-SAT es equivalente a:

$$X \Rightarrow \neg Y, \qquad Y \Rightarrow \neg X$$

Esto se aplica en dos casos:

- **Reutilización de polvo:** si dos opciones usan el mismo polvo, son incompatibles.
- **Estabilidad:** si una opción en la semana $i$ exige más neutralizantes de los que aporta una opción en la semana $i+1$, esa pareja también es incompatible.

# C. El Circo de la Luna — Grafo de implicaciones

## Nodos

Para cada semana $i$ creamos dos nodos:

$$A_i \qquad \text{y} \qquad B_i$$

que representan elegir el espectáculo A o B esa semana.

# C. El Circo de la Luna — Grafo de implicaciones

## Nodos

Para cada semana $i$ creamos dos nodos:

$$A_i \qquad \text{y} \qquad B_i$$

que representan elegir el espectáculo A o B esa semana.

## Aristas

Cada incompatibilidad

$$\neg(X \wedge Y)$$

se traduce en dos implicaciones:

$$X \Rightarrow \neg Y, \qquad Y \Rightarrow \neg X$$

por tanto añadimos dos aristas al grafo.

# C. El Circo de la Luna — Grafo de implicaciones

## Nodos

Para cada semana $i$ creamos dos nodos:

$$A_i \qquad \text{y} \qquad B_i$$

que representan elegir el espectáculo A o B esa semana.

## Aristas

Cada incompatibilidad

$$\neg(X \wedge Y)$$

se traduce en dos implicaciones:

$$X \Rightarrow \neg Y, \qquad Y \Rightarrow \neg X$$

por tanto añadimos dos aristas al grafo.

**Ejemplo:** si no podemos elegir $A_i$ y $B_j$ a la vez, añadimos:

$$A_i \Rightarrow A_j, \qquad B_j \Rightarrow B_i$$

porque $\neg(B_j) = A_j$ y $\neg(A_i) = B_i$.

## C. El Circo de la Luna — Satisfacibilidad con Kosaraju

Una vez construido el grafo de implicaciones, comprobamos si la instancia es satisfacible usando **componentes fuertemente conexas (SCC)**.

## C. El Circo de la Luna — Satisfacibilidad con Kosaraju

Una vez construido el grafo de implicaciones, comprobamos si la instancia es satisfacible usando **componentes fuertemente conexas (SCC)**.

### Criterio clásico de 2-SAT

La instancia es satisfacible si para ninguna semana $i$ están en la misma SCC los dos literales opuestos:

$$A_i \quad \text{y} \quad B_i$$

# C. El Circo de la Luna — Satisfacibilidad con Kosaraju

Una vez construido el grafo de implicaciones, comprobamos si la instancia es satisfacible usando **componentes fuertemente conexas (SCC)**.

## Criterio clásico de 2-SAT

La instancia es satisfacible si para ninguna semana $i$ están en la misma SCC los dos literales opuestos:

$$A_i \quad \text{y} \quad B_i$$

¿Por qué? Porque si ambos caen en la misma componente, entonces:

$$A_i \Rightarrow B_i \qquad \text{y} \qquad B_i \Rightarrow A_i$$

es decir, elegir uno obliga al otro y viceversa, lo cual es imposible.

# C. El Circo de la Luna — Satisfacibilidad con Kosaraju

Una vez construido el grafo de implicaciones, comprobamos si la instancia es satisfacible usando **componentes fuertemente conexas (SCC)**.

## Criterio clásico de 2-SAT

La instancia es satisfacible si para ninguna semana $i$ están en la misma SCC los dos literales opuestos:

$$A_i \quad \text{y} \quad B_i$$

¿Por qué? Porque si ambos caen en la misma componente, entonces:

$$A_i \Rightarrow B_i \qquad \text{y} \qquad B_i \Rightarrow A_i$$

es decir, elegir uno obliga al otro y viceversa, lo cual es imposible.
Aplicamos **Kosaraju**, que encuentra todas las SCC del grafo en tiempo lineal:

$$\mathcal{O}(V + E)$$

donde aquí:

- $V = 2k$ nodos,
- $E$ es el número de implicaciones añadidas.

# C. El Circo de la Luna — Respuesta final

Ya sabemos comprobar si un prefijo de longitud $k$ es válido.

# C. El Circo de la Luna — Respuesta final

Ya sabemos comprobar si un prefijo de longitud $k$ es válido.

## Monotonía

Si se pueden programar las primeras $k$ semanas, entonces también se puede programar cualquier prefijo menor.

# C. El Circo de la Luna — Respuesta final

Ya sabemos comprobar si un prefijo de longitud $k$ es válido.

**Monotonía**

Si se pueden programar las primeras $k$ semanas, entonces también se puede programar cualquier prefijo menor.

Por tanto, la respuesta es monótona y podemos buscar el mayor $k$ mediante **búsqueda binaria**.

# C. El Circo de la Luna — Respuesta final

Ya sabemos comprobar si un prefijo de longitud $k$ es válido.

## Monotonía

Si se pueden programar las primeras $k$ semanas, entonces también se puede programar cualquier prefijo menor.

Por tanto, la respuesta es monótona y podemos buscar el mayor $k$ mediante **búsqueda binaria**.

## Complejidad

Probamos $\mathcal{O}(\log M)$ valores de $k$.
Para cada uno:

- construimos el grafo de implicaciones;
- ejecutamos Kosaraju en tiempo lineal sobre ese grafo.

# Marcador Final

## Marcador final